

Recomendación local de citas académicas mediante aprendizaje automático

Reporte final – Entrega 3

Maestría en Inteligencia Artificial – Universidad de los Andes

Carlos Duque · Carlos Caicedo · Jose Zambrana · John Arias

Proyecto: Desarrollo de Soluciones

22 de septiembre de 2026

1. Problema, datos y alcance

Encontrar una referencia adecuada para respaldar una afirmación académica puede convertirse en una tarea costosa cuando el autor debe buscar entre miles de publicaciones relacionadas con un mismo tema. La recomendación local de citas busca reducir ese esfuerzo utilizando como punto de partida el fragmento de texto en el que se necesita incorporar una referencia.

Este problema ha sido abordado mediante modelos que utilizan el contexto local de la cita para recuperar y ordenar artículos relevantes. En particular, Gu, Gao y Hahnloser (2022) proponen una arquitectura de recomendación local de citas que combina recuperación y reordenamiento de candidatos.

Pregunta de negocio: ¿Cómo apoyar a un autor de textos académicos en la identificación de artículos potencialmente relevantes para citar utilizando el contexto textual local?

El prototipo desarrollado recibe un fragmento académico en inglés y devuelve una lista ordenada de artículos candidatos. Para cada resultado se presentan el identificador del documento, el título, el resumen y las puntuaciones utilizadas por el recomendador.

El proyecto trabaja con el conjunto `custom` (ACL-200), proveniente del repositorio *Local-Citation-Recommendation* publicado por Gu (2022). El corpus utilizado contiene 63.768 contextos de cita y 19.776 artículos candidatos. Las particiones empleadas incluyen 30.390 consultas de entrenamiento, 9.381 de validación y 9.585 de prueba.

Los archivos de datos se administran con DVC a través de `data/raw.dvc` y `data/processed.dvc`. De esta forma, Git se mantiene enfocado en el código, la configuración y la documentación, mientras DVC conserva el versionamiento de los conjuntos utilizados durante los experimentos.

2. Arquitectura y modelo

La solución se organizó en dos etapas. Primero, TF-IDF recupera los artículos más parecidos al contexto de consulta. Después, un modelo supervisado vuelve a ordenar esos candidatos para intentar ubicar la cita correcta entre las primeras posiciones.

Contexto → TF-IDF → Top-100 → Regresión logística → Top-K → paquete → FastAPI → tablero

La primera etapa compara el contexto con los 19.776 artículos del corpus mediante similitud coseno y conserva los 100 candidatos con mayor puntaje.

Sobre esos candidatos se aplica una regresión logística que utiliza cinco características:

- similitud TF-IDF entre el contexto y el título;
- similitud TF-IDF entre el contexto y el resumen;
- longitud del contexto;
- longitud del título;
- longitud del resumen.

Para entrenar el reordenador se construyeron aproximadamente 91.170 pares contexto-artículo. Cada consulta aporta una cita positiva y dos ejemplos negativos.

Se probaron dos formas de seleccionar esos negativos. En la primera, `random`, los artículos incorrectos se escogen aleatoriamente. En la segunda, `hard`, se utilizan artículos incorrectos que TF-IDF considera muy parecidos al contexto.

Las siguientes capturas corresponden a las ejecuciones utilizadas para entrenar ambas variantes.

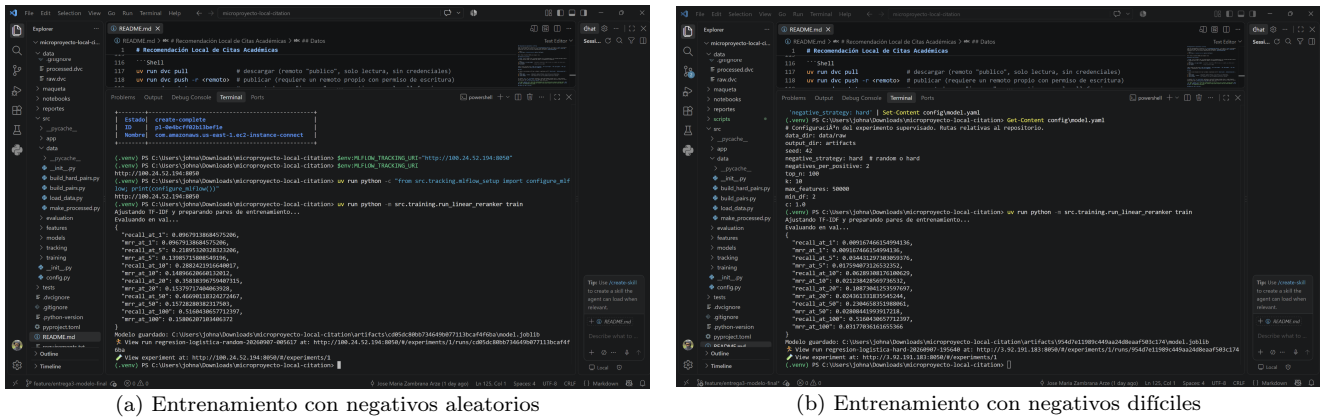


Figura 1: Evidencias de entrenamiento de las dos estrategias evaluadas.

3. Resultados experimentales

Las dos variantes se evaluaron con las mismas particiones de datos, la misma semilla y los mismos parámetros generales. La única diferencia controlada fue la estrategia utilizada para seleccionar los ejemplos negativos.

Los resultados principales fueron los siguientes:

Tabla 1: Comparación de los modelos sobre las 9.381 consultas de validación

Modelo	Recall@10	MRR@10	Recall@100
TF-IDF sin reordenar	0,2571	0,1339	0,5160
v1 · Random	0,2882	0,1490	0,5160
v1 · Hard	0,0629	0,0212	0,5160
v2 · Random	0,3793	0,2166	0,5160
v2 · Hard	0,2288	0,0995	0,5160

El *Recall@100* se mantiene en 0,5160 en las cinco filas. Esa invariancia comprueba que el reordenador opera efectivamente como segunda etapa –recibe el mismo conjunto recuperado y solo altera el orden– y marca a la vez el techo del sistema: mientras la primera etapa no recupere el artículo correcto, ningún reordenamiento puede corregirlo.

La línea base de comparación es 0,2571 / 0,1339 y no 0,2541 / 0,1249, porque excluye el artículo citante igual que lo hace el conjunto de candidatos que consume el reordenador. Compararlo contra la versión que sí lo incluye inflaría la mejora atribuida al modelo.

La diferencia entre las dos estrategias es considerable en las primeras posiciones del ranking. Con negativos aleatorios, Recall@10 alcanza 0,2882, mientras que con negativos difíciles baja a 0,0629. El mismo comportamiento aparece en MRR@10, que pasa de 0.1490 a 0.0212.

Las gráficas siguientes permiten observar esta diferencia de manera directa.

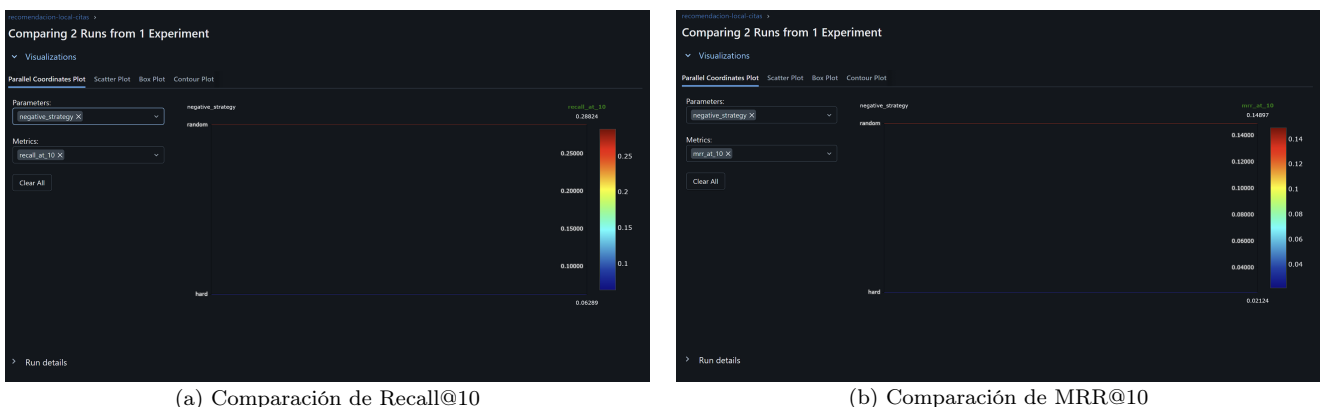


Figura 2: Desempeño de las estrategias Random y Hard en las primeras posiciones del ranking.

3.1 Segunda versión: ampliación de las características

La comparación anterior mostró que el cuello de botella estaba en la representación y no en el algoritmo. Entrenado con negativos difíciles, el reordenador aprendía un coeficiente de $-2,09$ sobre la similitud con el resumen: prefería lo *menos* parecido. No es un error de implementación, sino un ajuste fiel a su señal, porque el área bajo la curva ROC de positivos frente a negativos difíciles usando solo similitud coseno es $0,4053$, por debajo de $0,5$. Con cinco

características léxicas, separar un positivo de los cien candidatos más parecidos es una tarea que el vector no puede resolver.

La segunda versión amplía el vector a diez características. Las cinco nuevas se derivan de los metadatos que codifica el identificador del ACL Anthology –tipo de publicación, año y número, formato que cumplen los 19.776 artículos del corpus– y del conjunto de entrenamiento:

- `year_diff`: años entre el artículo citante y el candidato.
- `is_future`: candidato publicado después del citante, es decir una cita imposible.
- `same_venue`: coincidencia en el tipo de publicación.
- `title_overlap`: fracción del título presente en el contexto.
- `citation_prior`: logaritmo de las veces que el candidato se cita en entrenamiento.

La restricción temporal resultó la señal más valiosa. Antes de incorporarla, el 11,1 % de los cupos del *Top-100* se gastaba en artículos posteriores al citante, y de las 9.381 consultas de validación **ninguna** tiene un positivo posterior, de modo que la restricción es exacta y descartarlos no puede eliminar un acierto legítimo.

Los coeficientes de la variante ganadora confirman su peso: sobre características estandarizadas, `is_future` obtiene $-3,0790$ y `citation_prior` $+2,6889$, ambos por encima de `similarity_abstract` ($+2,4954$). El modelo aprendió la restricción temporal por sí solo, sin que se le impusiera como filtro.

El modelo seleccionado es la versión 2 con negativos aleatorios, por métricas de ranking y no por *accuracy*: frente a la línea base comparable mejora un 47,5 % en Recall@10 y un 61,8 % en MRR@10; frente a la primera versión, un 31,6 % y un 45,4 %. La ganancia es mayor en MRR que en Recall, que es justamente el comportamiento esperado de una segunda etapa: no encuentra artículos nuevos, sube los que ya estaban.

Con el vector ampliado, la variante de negativos difíciles mejora 3,6 veces en Recall@10 (de 0,0629 a 0,2288) pero sigue perdiendo. La suma de los coeficientes de similitud permanece negativa, de modo que las características nuevas compensan la señal invertida sin corregirla. Explotar negativos difíciles exigiría una representación densa, no más señales tabulares.

Recall@100 permanece alrededor de 0,516 en las dos configuraciones porque ambas reciben exactamente los mismos 100 candidatos recuperados por TF-IDF. Esto indica que la diferencia entre los experimentos se encuentra en el ordenamiento y no en la recuperación inicial.

El resultado de la estrategia **hard** también permitió identificar una limitación del modelo. Los negativos difíciles son artículos que ya presentan una gran similitud léxica con el contexto y, utilizando únicamente cinco características, el reordenador dispone de poca información adicional para distinguirlos correctamente.

Por este motivo se seleccionó la estrategia **random** como configuración del modelo empaquetado. La versión actual utiliza semilla 42, dos negativos por positivo, Top-100, $K = 10$, `max_features=50000`, `min_df=2` y $C = 1,0$.

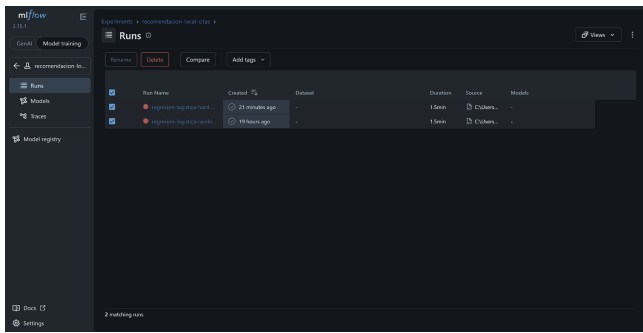
4. Seguimiento de experimentos con MLflow

Para llevar un registro común de los experimentos realizados por el equipo, el entrenamiento se integró con MLflow. Cada corrida conserva sus parámetros, métricas de ranking, tiempos de procesamiento y artefactos asociados.

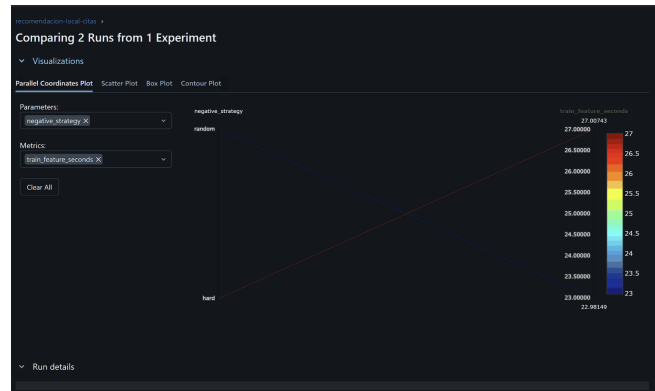
La configuración se centralizó en `src/tracking/mlflow_setup.py` para que todas las ejecuciones siguieran la misma estructura de seguimiento.

Este tipo de organización es coherente con las prácticas de MLOps, en las que se busca mantener trazabilidad sobre datos, experimentos, modelos y configuraciones durante el desarrollo de una solución de aprendizaje automático (Treveil & Dataiku Team, 2021).

Las siguientes capturas muestran las corridas Random y Hard registradas en MLflow y la comparación de sus tiempos de procesamiento.



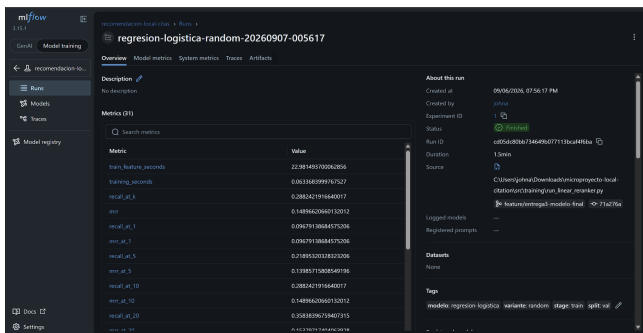
(a) Corridas Random y Hard registradas en MLflow



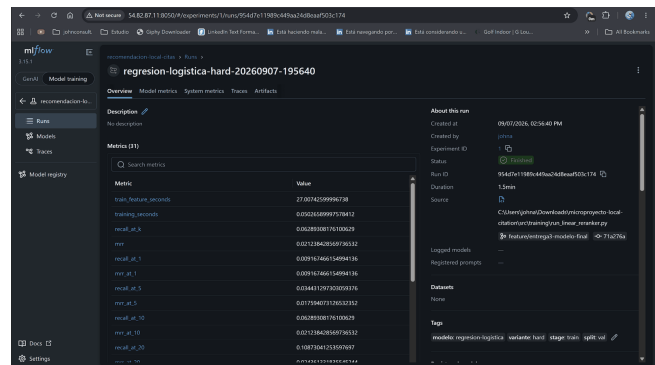
(b) Comparación del tiempo de generación de características

Figura 3: Seguimiento y comparación de los experimentos realizados.

Las siguientes capturas muestran el detalle de las métricas y los parámetros guardados para cada ejecución.



(a) Detalle de la corrida Random



(b) Detalle de la corrida Hard

Figura 4: Detalle de las dos variantes de entrenamiento almacenadas en MLflow.

El uso de MLflow permitió comparar las dos variantes sobre una misma base y conservar evidencia de los resultados utilizados para escoger la configuración final del modelo.

5. MLflow en AWS

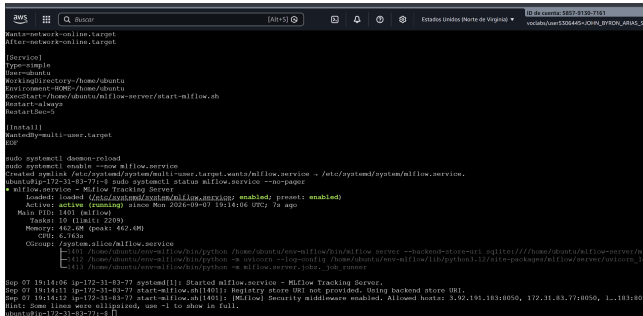
Para que el seguimiento de experimentos no dependiera de una sola máquina local, se instaló MLflow 3.15.1 en una instancia EC2 de AWS Academy con Ubuntu 24.04 LTS.

La instancia utiliza SQLite para guardar los metadatos de MLflow y un directorio local para los artefactos. El servicio escucha por el puerto 8050 y se configuró con `systemd` para que vuelva a iniciarse automáticamente cuando la máquina arranque.

Uno de los inconvenientes encontrados durante el trabajo con AWS Academy fue el cambio de la dirección IP pública después de detener y volver a iniciar la instancia. Para evitar tener que actualizar esta dirección manualmente cada vez, se creó el script `scripts/conectar_mlflow_aws.ps1`.

El script consulta la IP pública actual mediante AWS CLI, configura `MLFLOW_TRACKING_URI` y comprueba que el puerto 8050 sea accesible desde el equipo local.

Las siguientes imágenes muestran el servicio funcionando en EC2 y la conexión automática desde PowerShell.



(a) Servicio MLflow activo en EC2



(b) Detección de la IP y conexión desde el equipo local

Figura 5: Funcionamiento de la infraestructura de MLflow en AWS.

6. Empaquetamiento e integración del modelo

Otro avance de esta entrega fue convertir el recomendador en una librería Python instalable. El paquete se encuentra en `model-package/` y se distribuye con el nombre `modelo-citas`. La versión utilizada actualmente es la 0.1.0.

El wheel incluye el recuperador TF-IDF, el reordenador lineal, la configuración, el modelo entrenado y la información necesaria para generar las recomendaciones.

La interfaz pública del paquete puede utilizarse de esta forma:

```
from modelo_citas import make_prediction

resultado = make_prediction(
    context="Recent work on neural machine translation with attention",
    top_k=5,
)
```

Para comprobar que el paquete no funcionara únicamente por encontrarse dentro del repositorio, se construyó el wheel y se instaló en un entorno virtual limpio. Desde ese entorno se importó `modelo_citas` directamente desde `site-packages` y se ejecutó una predicción real.

También se ejecutaron las pruebas específicas del paquete mediante Tox. Después de integrar el cambio con la rama principal, se volvió a ejecutar la suite completa del proyecto.

Resultado de las pruebas: 96 pruebas aprobadas.

La integración del modelo empaquetado no produjo fallos funcionales en la suite del proyecto. Permanecen advertencias de deprecación relacionadas con NumPy/Joblib que no impiden la ejecución actual.

7. API y tablero

El paquete `modelo-citas` se integró con el backend desarrollado en FastAPI. Al iniciar la aplicación, el backend carga el modelo empaquetado y queda disponible para responder las consultas enviadas desde la API o desde el tablero.

Los endpoints principales son:

Método	Ruta	Función
GET	<code>/api/estado</code>	Estado del modelo
POST	<code>/api/recomendar</code>	Generar recomendaciones
GET	<code>/api/insights</code>	Información analítica
GET	<code>/api/ejemplo</code>	Ejemplo real del corpus
GET	<code>/docs</code>	Documentación Swagger/OpenAPI

Las pruebas realizadas desde Swagger devolvieron código HTTP 200 y resultados reales para el endpoint de recomendación. El endpoint `/api/estado` también confirmó que el modelo cargado corresponde a TF-IDF más reordenador lineal, versión 0.1.0, sobre un catálogo de 19.776 artículos.

El tablero se encuentra en `src/app/`. Desde la interfaz, el usuario puede escribir un contexto, escoger cuántas recomendaciones desea recibir y consultar el ranking generado. Al seleccionar un artículo también puede revisar su resumen.

Además del recomendador, la interfaz presenta tres apartados de análisis: estudio de los datos, desempeño del modelo y diagnóstico de negativos. Así, desde una misma interfaz es posible probar el recomendador y consultar información sobre los datos y el desempeño obtenido durante los experimentos.

8. Uso, instalación y reproducibilidad

8.1 Uso del prototipo

El flujo para utilizar el tablero es el siguiente:

1. abrir la aplicación en el navegador;
2. escribir o pegar un fragmento académico en inglés;
3. escoger el número de recomendaciones;
4. pulsar *Recomendar citas*;
5. revisar los artículos recuperados y sus resúmenes.

También puede utilizarse la opción *Usar un ejemplo real* para cargar una consulta conocida del conjunto de datos y observar el comportamiento del recomendador.

Los resultados deben entenderse como apoyo a la búsqueda bibliográfica. La pertinencia de una referencia debe revisarse antes de utilizarla en un documento académico.

8.2 Instalación

El flujo principal para preparar el proyecto es:

```
git clone https://github.com/Byron971/microproyecto-local-citation.git
cd microproyecto-local-citation
uv sync
uv run dvc pull
uv run pytest
uv run tox -c model-package -e train
uv sync --reinstall-package modelo-citas
uv run tablero
```

Una vez iniciado el servicio, el tablero queda disponible en <http://127.0.0.1:8000> y la documentación de la API en <http://127.0.0.1:8000/docs>.

La reproducibilidad del proyecto se apoya en Git y GitHub para el código, DVC para los datos, MLflow para los experimentos, uv para las dependencias, Tox para las pruebas del paquete y pytest para la suite general.

9. Estado actual de la Entrega 3

Hasta el momento se han completado los componentes relacionados con los datos, la experimentación, la selección del modelo, el empaquetamiento y la integración local con la API y el tablero.

Componente	Estado
Repositorio Git y colaboración	Realizado
Datos versionados con DVC	Realizado
Modelos y comparación experimental	Realizado
Seguimiento con MLflow	Realizado
Modelo seleccionado	Realizado
Modelo empaquetado	Realizado
API con modelo empaquetado	Realizado
Tablero integrado	Realizado
Pruebas automatizadas	Suite incluida y ampliada
Manual de usuario e instalación	Incluido
Dockerización	Realizado
Despliegue mediante contenedores	Realizado
Despliegue en AWS	Realizado
Validación final del despliegue	Verificada desde entorno limpio
Video final	Pendiente
Reporte final de aportes	Realizado

Tabla 2: Estado de los componentes trabajados hasta el momento.

La validación del despliegue se realizó sobre un clon nuevo del repositorio, sin credenciales de AWS, reproduciendo el recorrido de una persona ajena al equipo: `dvc pull` recuperó los 117,6 MB de datos sin pedir autenticación, las pruebas pasaron, el paquete del modelo se instaló y entrenó, y `docker compose build` construyó las dos imágenes desde cero en 287 segundos. La API quedó en estado *healthy* a los diez segundos y el tablero devolvió, a través de Nginx, el mismo ranking que la ejecución local. El procedimiento completo y sus tiempos están en [docs/manual_instalacion.md](#).

Para cerrar la Entrega 3 queda grabar el video de síntesis, incorporar su enlace en la entrega y realizar la revisión final del PDF generado.

10. Trabajo en equipo

El trabajo se ha organizado mediante ramas, issues, commits y Pull Requests. Los cambios de mayor impacto se revisan antes de incorporarse a `main`. El repositorio también cuenta con `CODEOWNERS`, lo que permite solicitar revisión a otros integrantes antes de aceptar determinados cambios.

Durante esta fase se revisó especialmente el Pull Request que incorporó `model-package`. Antes de aprobarlo se comprobaron el entrenamiento, las pruebas del paquete, la construcción del wheel, su instalación en un entorno limpio y la integración con FastAPI y el tablero.

Integrante	Aportes principales
John Arias	Comparación Random vs. Hard y evidencias de MLflow; automatización de conexión a MLflow en AWS; preparación reproducible de DVC; integración del tablero con la API; dockerización de API y tablero, Docker Compose y documentación del despliegue en AWS; manual de usuario y consolidación del reporte. Para el corte del 20 de septiembre coordinó además la evaluación global, integró el pipeline de métricas, los clientes de modelos comerciales/open-weight y realizó la primera validación manual por parte del equipo sobre 20 casos piloto.
Carlos Duque	Desarrollo de la maqueta inicial en HTML y FastAPI; implementación de una aplicación simple para servir el tablero y predicciones; empaquetamiento del modelo como <code>modelo-citas</code> ; construcción del paquete instalable y apoyo en la integración del backend/tablero.
José Zambrana	Desarrollo del reordenador supervisado, extracción de características, regresión logística, generación de negativos difíciles, evaluación mediante Recall@K y MRR y documentación de resultados; apoyo en DVC y actualización de evidencias de MLflow. También diseñó y versionó los tres prompts de función de cita (zero-shot genérico, zero-shot detallado y few-shot), el conjunto piloto y el análisis de estabilidad y errores de formato.
Carlos Caicedo	Desarrollo y evaluación de la línea base TF-IDF; configuración inicial de MLflow y remotos DVC; diagnóstico de negativos; implementación y evaluación del reordenador versión 2 con características de metadatos; documentación del modelo final; creación y verificación del manual de instalación y auditoría desde entorno limpio, incluida evidencia de Docker y despliegue.

Tabla 3: Resumen de los aportes realizados hasta esta etapa del proyecto.

La tabla resume contribuciones verificables en el historial del repositorio, issues y Pull Requests. El video final debe mantener participación identificable de los integrantes y reflejar esta distribución de trabajo.

Como preparación adicional para la evaluación de función de cita se versionaron tres estrategias de *prompting*, el parser común de nueve puntajes y el pipeline de métricas con Precision, Recall, F1 Macro/Micro y matrices de confusión. También se completó una primera validación manual por parte del equipo sobre 20 casos piloto. Este conjunto se conserva como evaluación provisional: todavía requiere una segunda validación manual independiente, cálculo de acuerdo interanotador y reconciliación antes de denominarlo *Test Gold* definitivo.

11. Conclusiones

La comparación realizada permitió escoger una configuración concreta para el reordenador. La versión 2 con negativos aleatorios alcanzó Recall@10 de 0,3793 y MRR@10 de 0,2166, mientras que la versión 2 con negativos difíciles obtuvo 0,2288 y 0,0995, respectivamente. Por esta razón se seleccionó la versión 2 con negativos aleatorios como modelo final.

El experimento también permitió identificar una limitación importante. Al mantenerse Recall@100 cerca de 0,516 en ambas variantes, aproximadamente la mitad de las citas correctas quedan fuera del conjunto inicial de 100 candidatos. Por esta razón, una mejora futura no debería concentrarse únicamente en el reordenador, sino también en la etapa de recuperación.

Además de mejorar el modelo, durante esta entrega se avanzó en la construcción de una solución que pueda reproducirse y reutilizarse fuera del entorno de experimentación. Los datos están versionados con DVC, los experimentos quedan registrados en MLflow, el modelo puede instalarse como wheel y ya es consumido por FastAPI y por el tablero.

La contenerización y el despliegue quedaron completados y verificados: las imágenes de la API y del tablero se construyen desde cero sin intervención manual, y la solución se probó tanto en local como sobre AWS. La reproducción desde un clon limpio confirmó que una persona ajena al equipo puede levantar el sistema completo sin credenciales propias.

Para cerrar la Entrega 3 queda pendiente el video de síntesis y la actualización final de los aportes del equipo. En la extensión de clasificación de función de cita todavía se requiere una segunda validación manual independiente y las corridas reales de los modelos configurados antes de reportar resultados definitivos. De cara a la continuación del trabajo, el techo del recomendador está en la primera etapa: con un Recall@100 de 0,5160, la mitad de las consultas no tiene solución posible por reordenamiento, de modo que el margen de mejora está en la recuperación de candidatos –por ejemplo mediante representaciones densas– antes que en el reordenador.

Referencias

- Gu, N. (2022). *Local-Citation-Recommendation* [Computer software]. GitHub. <https://github.com/nianlonggu/Local-Citation-Recommendation>
- Gu, N., Gao, Y., & Hahnloser, R. H. R. (2022). Local citation recommendation with hierarchical-attention text encoder and SciBERT-based reranking. *Advances in Information Retrieval*, 274–288. Springer. https://doi.org/10.1007/978-3-030-99736-6_19
- Treveil, M., & Dataiku Team. (2021). *Introducing MLOps: How to scale machine learning in the enterprise*. O'Reilly Media.